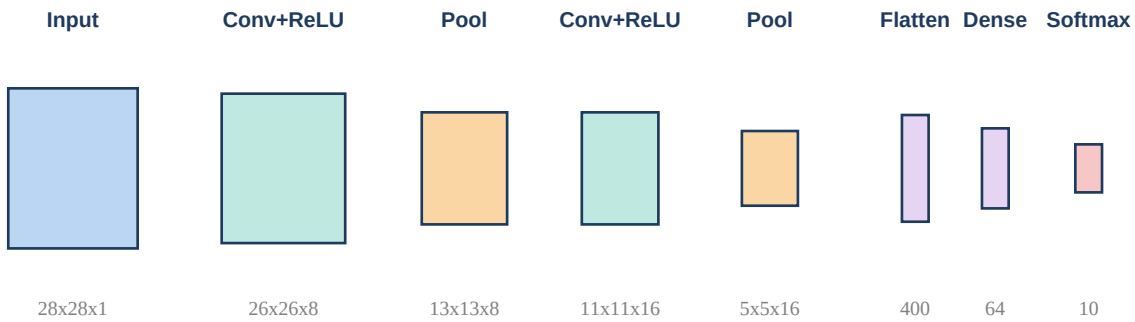
Convolutional Neural Networks

Complete Interview Notes with Worked Examples



Feature extraction (conv + pool) -> Classification (dense)

Inside
1. Why CNNs 2. Images as numbers 3. Convolution (worked example) 4. Stride, padding, output size
5. Multi-channel convolution & parameter counting 6. ReLU & pooling 7. Full architecture walkthrough
8. Backprop in CNNs 9. Receptive field, 1x1 conv 10. Famous architectures 11. Training tricks & transfer learning
12. Detection & segmentation 13. Code (Keras + PyTorch) 14. 30 interview Q&As 15. Cheat sheet

1. What is a CNN and why do we need it?

A **Convolutional Neural Network (CNN)** is a neural network specialised for grid-like data (images, video, audio spectrograms). Instead of connecting every input to every neuron, it slides small learnable **filters (kernels)** across the input to detect local patterns such as edges, textures and shapes, then combines them into higher-level features.

One-line interview answer: *A CNN uses convolution layers with shared, small filters to automatically learn a hierarchy of spatial features, followed by pooling and dense layers for the final prediction.*

Why a plain ANN fails on images

- **Too many parameters:** a 224x224 colour image has 224*224*3 = 150,528 inputs. One dense layer of 1,000 neurons needs about **150 million weights**. Overfitting and memory issues follow.
- **Ignores spatial structure:** flattening the image destroys the fact that neighbouring pixels are related.
- **Not translation-invariant:** a cat in the top-left and a cat in the bottom-right look like completely different inputs to a dense layer.

What CNNs fix: three core ideas

Idea	Meaning	Benefit
Local connectivity	Each neuron looks only at a small patch (e.g. 3x3)	Far fewer weights; captures local patterns
Weight sharing	The same filter is reused at every position	Parameters independent of image size; detects a feature anywhere
Hierarchy	Early layers: edges. Middle: textures/parts. Deep: objects	Automatic feature learning, no manual feature engineering

2. Images as Numbers

A grayscale image is a 2D matrix of pixel intensities (0 = black, 255 = white). A colour image is a 3D tensor of shape **Height x Width x Channels** (3 channels: R, G, B). A batch adds one more dimension: (N, H, W, C) in TensorFlow or (N, C, H, W) in PyTorch.

Image	Shape	Number of values
28x28 grayscale (MNIST)	28 x 28 x 1	784
32x32 colour (CIFAR-10)	32 x 32 x 3	3,072
224x224 colour (ImageNet)	224 x 224 x 3	150,528

Always **normalize** pixels (divide by 255, or subtract mean and divide by std) before training for faster, more stable convergence.

3. The Convolution Operation (Worked Example)

A **filter** (kernel) is a small matrix of learnable weights. To convolve, slide the filter over the image; at each position multiply element-wise with the patch under it and **sum** the results into one output number. The grid of outputs is a **feature map**. (Strictly, deep-learning libraries compute *cross-correlation* - the filter is not flipped - but everyone calls it convolution.)

Example: detecting vertical edges

Input image 5x5 (1 = bright, 0 = dark) and a 3x3 vertical-edge filter (left column positive, right column negative):

Image 5x5

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

***** **Filter 3x3**

1	0	-1
1	0	-1
1	0	-1

= **Feature map 3x3**

-2	0	2
-3	-2	2
-3	-1	2

First output value (top-left patch):

```
patch = [[1, 1, 1], [0, 1, 1], [0, 0, 1]]
sum = (1*1) + (1*0) + (1*-1) + (0*1) + (1*0) + (1*-1) + (0*1) + (0*0) + (1*-1)
    = -2
```

The filter then slides one step right and repeats. With a 5x5 input, 3x3 filter, stride 1 and no padding, we get a **3x3** output (formula in the next section). Large positive/negative values mark where a vertical brightness change (edge) exists; values near 0 mean a flat region.

Key properties

- **Filters are learned:** we do not hand-design the edge filter; backprop discovers useful ones.
- **Weight sharing:** this one filter has just 9 weights (+1 bias), no matter how large the image is.
- **Translation equivariance:** if the object shifts, the feature map shifts by the same amount.

Interview tip: Say *equivariance* for convolution (shifting input shifts the feature map) and *invariance* for pooling / the final classifier (small shifts do not change the output). Many candidates mix these up.

4. Stride, Padding and Output Size

- **Stride (S):** how many pixels the filter moves each step. Stride 2 halves the spatial size (downsampling).
- **Padding (P):** adding zeros around the border. **'valid'** = no padding (output shrinks). **'same'** = pad so output size equals input size (for stride 1, P = (F-1)/2).

Why pad? Without it the image shrinks every layer and border pixels are used less than centre pixels.

Output size formula: O = floor((N + 2P - F) / S) + 1 where N = input size, F = filter size, P = padding, S = stride.

N	F	P	S	Calculation	Output
5	3	0	1	(5 + 0 - 3)/1 + 1	3
5	3	1	1	(5 + 2 - 3)/1 + 1	5 (same)
28	5	0	1	(28 - 5)/1 + 1	24
32	3	1	2	floor((32 + 2 - 3)/2) + 1	16
224	7	3	2	floor((224 + 6 - 7)/2) + 1	112

5. Multi-Channel Convolution and Parameter Counting

Real inputs have several channels (RGB = 3). A filter must have the **same depth as the input**: a 3x3 filter on an RGB image is really 3x3x3 = 27 weights. It produces **one** 2D feature map by summing across channels. To get C_out feature maps we use C_out different filters.

> **Parameters in a conv layer: (F x F x C_in + 1) x C_out** (the +1 is the bias of each filter).

Layer	Calculation	Parameters
3x3 conv, 1 in, 8 out	(3*3*1 + 1) * 8	80
3x3 conv, 3 in, 16 out	(3*3*3 + 1) * 16	448
3x3 conv, 64 in, 128 out	(3*3*64 + 1) * 128	73,856
Dense 400 -> 64	400*64 + 64	25,664

Compare: a dense layer from a 28x28 image (784 inputs) to 8 neurons needs 784*8 + 8 = 6,280 weights and gets *one* value per neuron. The conv layer above uses 80 parameters and produces 8 full 26x26 feature maps. Parameter count does not depend on the image size - that is the power of weight sharing.

Output channels = number of filters. Each filter specialises: one may detect horizontal edges, another a colour blob, another a texture.

6. Activation (ReLU) and Pooling

ReLU after convolution

After the convolution we apply **ReLU: max(0, x)** to add non-linearity. Applied to our feature map:

-2	0	2		0	0	2
-3	-2	2	**-- ReLU -->**	0	0	2
-3	-1	2		0	0	2

Pooling

Pooling **downsamples** each feature map independently (no learnable parameters). It shrinks computation, enlarges the receptive field and gives small-shift invariance. Typical: 2x2 window, stride 2, halving height and width.

Input 4x4

1	3	2	4
5	6	1	2
7	2	9	0
3	4	1	8

Max pool 2x2

6	4
7	9

Avg pool 2x2

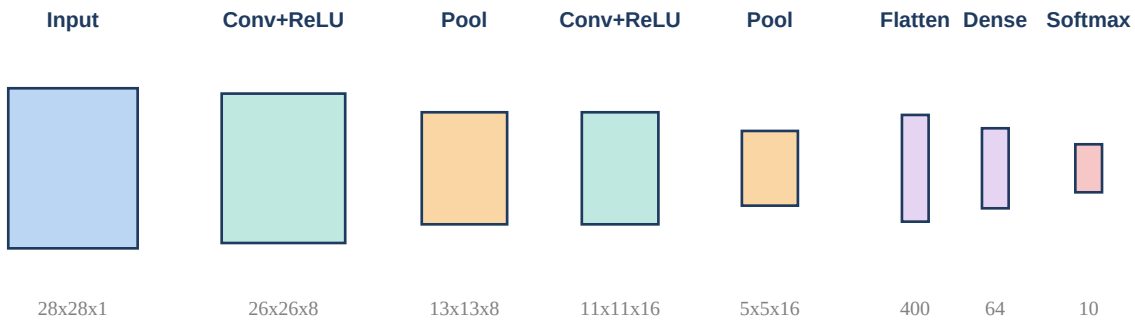
3.75	2.25
4.00	4.50

Pooling	What it does	When used
Max pooling	Keeps the strongest activation in each window	Default in classic CNNs; keeps the most salient feature
Average pooling	Averages the window	Smoother; used in some architectures
Global average pooling	Averages each whole feature map to one number	Replaces flatten + big dense layers (ResNet); fewer parameters, less overfitting

Modern designs often replace pooling with a **strided convolution** (learned downsampling).

7. Full CNN Architecture Walkthrough

A standard CNN is: **[Conv -> ReLU -> Pool] x N -> Flatten -> Dense -> Softmax**. Convolution blocks extract features; dense layers classify. Here is a small digit classifier (MNIST, 10 classes), tracing every shape and parameter count:



Layer	Output shape	Parameters	Calculation
Input	28 x 28 x 1	0	grayscale image
Conv 3x3, 8 filters, valid	26 x 26 x 8	80	(3*3*1+1)*8
ReLU	26 x 26 x 8	0	element-wise
MaxPool 2x2	13 x 13 x 8	0	26/2
Conv 3x3, 16 filters, valid	11 x 11 x 16	1,168	(3*3*8+1)*16
ReLU	11 x 11 x 16	0	element-wise
MaxPool 2x2	5 x 5 x 16	0	floor(11/2) = 5
Flatten	400	0	5*5*16
Dense 64 + ReLU	64	25,664	400*64+64
Dense 10 + Softmax	10	650	64*10+10
Total		**27,562**	

Observe the pattern: **spatial size shrinks** (28 -> 13 -> 5) while **depth grows** (1 -> 8 -> 16). The network trades spatial resolution for richer feature channels. Note that almost all parameters (25,664 of 27,562) sit in the dense layer, which is why modern networks use global average pooling.

What each depth learns

Layer depth	Learns	Example
Early	Edges, corners, colours	Lines at different angles
Middle	Textures, patterns, parts	Eyes, wheels, fur
Deep	Whole objects / semantics	Face, car, digit '7'

8. Training and Backpropagation in CNNs

Training is the same loop as any neural network: forward pass, compute loss (categorical cross-entropy for classification), backward pass, update with an optimizer (Adam/SGD+momentum). CNN-specific points:

- **Shared weights:** a filter is used at many positions, so its gradient is the **sum** of gradients from all positions where it was applied.
- **Gradient w.r.t. a filter** is itself a convolution of the input with the upstream gradient; the gradient w.r.t. the input is a convolution with the flipped filter (a 'transposed' convolution).

- **Max pooling backward:** the gradient flows only to the position that held the maximum; the others get 0. **Average pooling:** gradient is split equally.
- **ReLU backward:** gradient passes where input > 0 and is blocked elsewhere.

9. Receptive Field, 1x1 Convolution, Dilation

Receptive field

The **receptive field** of a neuron is the region of the original input that influences it. Each deeper layer sees more context. Stacking two 3x3 convs (stride 1) gives a 5x5 receptive field; three give 7x7.

Why VGG stacks 3x3 filters instead of one 7x7: three 3x3 layers cover the same 7x7 area, but with $3*(3*3*C*C) = 27C^2$ weights versus $49C^2$ for one 7x7, plus three non-linearities instead of one (more expressive).

1x1 Convolution

A 1x1 filter looks at one pixel but across all channels. It acts like a small dense layer applied at every pixel. Uses: **reduce or expand the number of channels** cheaply (bottleneck in Inception/ResNet), mix channel information, and add non-linearity. Example: 256 channels -> 64 channels costs (256+1)*64 = 16,448 parameters.

Other variants

Variant	Idea	Why use it
Dilated (atrous) conv	Gaps between filter taps	Larger receptive field without more parameters or downsampling (segmentation)
Depthwise separable conv	Depthwise (per channel) 3x3 + 1x1 pointwise	About 8-9x fewer computations; used in MobileNet/Xception
Transposed conv	Learnable upsampling	Decoders, segmentation, GANs
Grouped conv	Split channels into groups	Fewer parameters (ResNeXt)

10. Famous CNN Architectures

Model (year)	Key contribution	Notes
LeNet-5 (1998)	First practical CNN for digits	Conv + avg pool + dense; ~60K params
AlexNet (2012)	Won ImageNet; deep CNN on GPUs	ReLU, dropout, data augmentation
VGG-16 (2014)	Depth with only 3x3 convs	Simple but ~138M params (heavy)
GoogLeNet / Inception (2014)	Parallel filters of different sizes, 1x1 bottlenecks	Only ~5M params, efficient
ResNet (2015)	**Skip (residual) connections**	Trains 50-152+ layers; solves degradation/vanishing gradient
DenseNet (2017)	Each layer connects to all later layers	Feature reuse, fewer params
MobileNet (2017)	Depthwise separable convs	Built for phones and edge devices
EfficientNet (2019)	Compound scaling of depth, width, resolution	Strong accuracy per FLOP

Residual connection: output = F(x) + x. The block only needs to learn the residual F(x) = H(x) - x, and the identity path lets gradients flow directly to earlier layers, which makes very deep networks trainable.

11. Training Tricks, Regularization and Transfer Learning

Technique	Purpose
Data augmentation	Random flips, crops, rotations, colour jitter: creates more varied data, a strong regularizer for images
Batch Normalization	Normalizes layer outputs per mini-batch; faster, more stable training, allows higher learning rates
Dropout	Mostly used in dense layers of a CNN (0.3-0.5); rarely after conv layers
L2 weight decay	Penalizes large weights
Early stopping	Stop when validation loss stops improving
Learning-rate schedule	Step decay, cosine annealing, warm-up
Optimizer	Adam (default), or SGD with momentum for best final accuracy

Transfer learning

Train a CNN from scratch only if you have lots of data. Otherwise take a model pre-trained on ImageNet (ResNet, EfficientNet, MobileNet) and adapt it:

Your data	Strategy
Small, similar to ImageNet	Freeze all conv layers, replace and train only the final classifier (feature extraction)
Medium, similar	Freeze early layers, fine-tune the last few conv blocks with a small learning rate
Large, different domain	Fine-tune the whole network (or train from scratch)

Why it works: early layers learn universal features (edges, textures) that transfer across tasks; only the later, task-specific layers need retraining.

12. Beyond Classification: Detection and Segmentation

Task	Output	Popular models	Metric
Classification	One label per image	ResNet, EfficientNet	Accuracy, top-5, F1
Object detection	Boxes + class labels	YOLO, SSD, Faster R-CNN	IoU, mAP
Semantic segmentation	Class label per pixel	U-Net, DeepLab, FCN	mIoU, Dice
Instance segmentation	Separate mask per object	Mask R-CNN	mAP on masks

IoU (Intersection over Union) = area of overlap / area of union between predicted and true box. IoU of 1 = perfect, 0 = no overlap; a detection is usually 'correct' if IoU >= 0.5. **Non-Max Suppression (NMS)** removes duplicate overlapping boxes, keeping the highest-confidence one.

CNN is also used for medical imaging, OCR, face recognition, self-driving perception, 1D CNNs for time series/audio and text classification.

13. Code Examples

A. Keras: the exact network from section 7

```
from tensorflow import keras
from tensorflow.keras import layers

model = keras.Sequential([
    layers.Input(shape=(28, 28, 1)),
    layers.Conv2D(8, (3, 3), activation="relu"),     # -> 26x26x8   (80 params)
    layers.MaxPooling2D((2, 2)),                     # -> 13x13x8
    layers.Conv2D(16, (3, 3), activation="relu"),    # -> 11x11x16  (1,168 params)
    layers.MaxPooling2D((2, 2)),                     # -> 5x5x16
    layers.Flatten(),                                # -> 400
    layers.Dense(64, activation="relu"),             # 25,664 params
    layers.Dropout(0.3),
    layers.Dense(10, activation="softmax"),          # 650 params
])
model.summary()                                      # total 27,562

model.compile(optimizer="adam", loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
(X_tr, y_tr), (X_te, y_te) = keras.datasets.mnist.load_data()
X_tr = X_tr[..., None] / 255.0;  X_te = X_te[..., None] / 255.0
model.fit(X_tr, y_tr, epochs=10, batch_size=64, validation_split=0.1)
print(model.evaluate(X_te, y_te))                    # about 98-99% accuracy
```

B. PyTorch version

```
import torch.nn as nn

class SmallCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 8, 3), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(8, 16, 3), nn.ReLU(), nn.MaxPool2d(2))
        self.classifier = nn.Sequential(
            nn.Flatten(), nn.Linear(16*5*5, 64), nn.ReLU(),
            nn.Dropout(0.3), nn.Linear(64, 10))   # softmax is inside CrossEntropyLoss
    def forward(self, x):                          # x: (N, 1, 28, 28)
        return self.classifier(self.features(x))
```

C. Transfer learning with a pre-trained ResNet50 (Keras)

```
base = keras.applications.ResNet50(weights="imagenet", include_top=False,
                                   input_shape=(224, 224, 3))
base.trainable = False                             # freeze pre-trained layers
model = keras.Sequential([base, layers.GlobalAveragePooling2D(),
                          layers.Dropout(0.3), layers.Dense(5, activation="softmax")])
model.compile(optimizer=keras.optimizers.Adam(1e-3),
              loss="sparse_categorical_crossentropy", metrics=["accuracy"])
# Later: base.trainable = True; recompile with lr=1e-5 to fine-tune
```

Practical checklist: normalize inputs, use data augmentation, start with a pre-trained model, monitor validation loss, check the shapes with *model.summary()*, and make sure the last activation matches the loss.

CNN vs ANN vs RNN vs Vision Transformer

	ANN / MLP	CNN	RNN / LSTM	Vision Transformer
Data	Tabular	Images, grids	Sequences	Images (as patches), text
Connectivity	Fully connected	Local + shared weights	Recurrent state	Global self-attention
Spatial bias	None	Strong (locality, translation)	None	Weak; needs lots of data
Parameters	Many	Few (sharing)	Moderate	Many
Data hunger	Moderate	Moderate	Moderate	High

14. Interview Questions and Answers

Q1. What is a CNN?

A neural network that uses convolution layers with small shared filters to learn spatial feature hierarchies from grid-like data, usually followed by pooling and dense layers.

Q2. Why use CNN instead of an ANN for images?

Dense layers need enormous numbers of weights for images, ignore spatial structure and are not translation-aware. CNNs use local connectivity and weight sharing, so they need far fewer parameters and detect features anywhere.

Q3. Explain the convolution operation.

Slide a small filter across the input; at each position multiply element-wise with the patch and sum to get one output value. The outputs form a feature map.

Q4. What is a filter/kernel? Is it hand-designed?

A small matrix of learnable weights (e.g. 3x3). Filters are learned by backpropagation, not hand-designed.

Q5. What is a feature map?

The output of applying one filter to the input (after activation). A layer with K filters outputs K feature maps.

Q6. What are stride and padding?

Stride is the step size of the filter; larger stride downsamples. Padding adds border zeros: valid = none, same = keeps output size.

Q7. Give the output size formula.

O = floor((N + 2P - F)/S) + 1. Example: N=32, F=3, P=1, S=1 gives 32.

Q8. How do you compute the parameters of a conv layer?

(F*F*C_in + 1) * C_out. Example: 3x3, 3 in, 16 out = (27+1)*16 = 448.

Q9. What is weight sharing and why does it matter?

The same filter is applied at every location, so the parameter count is independent of image size and a feature is detected wherever it appears.

Q10. What does pooling do and does it have parameters?

It downsamples feature maps (max or average), cuts computation, enlarges receptive field, adds small-shift invariance. It has no learnable parameters.

Q11. Max pooling vs average pooling?

Max keeps the strongest response (good for presence of features); average smooths. Global average pooling replaces dense layers in modern nets.

Q12. Why use ReLU in CNNs?

Cheap, non-saturating for positive values (less vanishing gradient) and sparse activations; speeds up training.

Q13. Translation invariance vs equivariance?

Convolution is equivariant: shifting input shifts the feature map. Pooling and the final classifier give (approximate) invariance.

Q14. What is the receptive field?

The input region that affects a neuron's value. It grows with depth, kernel size, stride and pooling.

Q15. Why stack small 3x3 filters instead of a large one?

Same receptive field (two 3x3 = one 5x5) with fewer parameters and more non-linearities.

Q16. What is a 1x1 convolution used for?

Changing the number of channels cheaply (bottlenecks), mixing channel information and adding non-linearity at low cost.

Q17. What is a depthwise separable convolution?

A per-channel spatial convolution followed by a 1x1 pointwise convolution; much cheaper than a standard conv. Used in MobileNet.

Q18. What problem does ResNet solve and how?

Very deep plain networks get harder to train (degradation, vanishing gradients). Skip connections compute F(x) + x so gradients flow directly and blocks learn residuals.

Q19. How does backprop work for max pooling?

The gradient is routed only to the input position that was the maximum; all others receive zero.

Q20. How do you reduce overfitting in a CNN?

Data augmentation, dropout, L2, batch norm, early stopping, smaller model, more data, transfer learning.

Q21. What is batch normalization and where do you place it?

It normalizes activations with batch mean/variance then scales and shifts with learned parameters. Typically after conv and before (or after) the activation.

Q22. What is transfer learning and when do you use it?

Reusing a pre-trained model (e.g. ImageNet) and fine-tuning for your task. Best when your dataset is small.

Q23. Feature extraction vs fine-tuning?

Feature extraction freezes the pre-trained base and trains only a new head. Fine-tuning also unfreezes some or all base layers with a small learning rate.

Q24. What is data augmentation? Name examples.

Generating modified training samples: flip, rotate, crop, scale, brightness/contrast, noise, cutout/mixup. It improves generalization.

Q25. What is the vanishing gradient problem in deep CNNs?

Gradients shrink when multiplied through many layers. Mitigated by ReLU, good init (He), batch norm and residual connections.

Q26. What is IoU and mAP?

IoU = overlap / union of predicted and true boxes. mAP is the mean over classes of average precision across IoU thresholds; it is the standard detection metric.

Q27. Difference between classification, detection and segmentation?

Classification gives one label per image; detection gives boxes and labels; segmentation labels every pixel.

Q28. What is global average pooling?

It averages each feature map into a single value, producing a vector with one entry per channel. It removes the need for big dense layers and reduces overfitting.

Q29. Why is the input image size usually fixed for a CNN with dense layers?

The flatten layer must produce a fixed length vector for the dense layer. Fully convolutional networks / global pooling remove this restriction.

Q30. Can CNNs be used for non-image data?

Yes: 1D CNNs for time series, audio and text; 3D CNNs for video and volumetric medical scans.

Q31. Limitations of CNNs?

Limited global context, not rotation/scale invariant without augmentation, need large labelled data, can be fooled by adversarial noise, and struggle with relationships between distant parts (where Vision Transformers help).

15. One-Page Cheat Sheet

Concept	Remember
Conv output size	floor((N + 2P - F)/S) + 1
Conv parameters	(F*F*C_in + 1) * C_out
'same' padding (stride 1)	P = (F - 1)/2
Pooling	No parameters; 2x2 stride 2 halves H and W
Typical block	Conv -> (BatchNorm) -> ReLU -> Pool
Pattern	Spatial size shrinks, channels grow
Receptive field	Two 3x3 = 5x5, three 3x3 = 7x7
1x1 conv	Change channels cheaply; bottleneck
Residual block	output = F(x) + x
Output layer	Softmax + cross-entropy (multi-class), sigmoid + BCE (binary)
Small dataset	Transfer learning + augmentation
Detection metrics	IoU, mAP, NMS
Equivariance / invariance	Conv = equivariant; pooling = invariant

How to answer CNN questions in an interview: (1) one-line definition, (2) intuition with a tiny numeric example (like the 5x5 edge example), (3) formula for shapes or parameters, (4) trade-off or limitation, (5) tie it to a real project. Working a shape or parameter count out loud shows real understanding.